

DutyGraph Listen Intelligently

Participant Voice Capture V2 - Product, UX, Architecture, and Engineering Specification

Listen intelligently. Map the work.

1. Product objective

Rebuild the DutyGraph employee discovery experience so that responding to a private work-discovery request feels like briefing a capable new teammate, not recording a deposition, completing a survey, or training an AI replacement.

1. Open the private DutyGraph link.
2. Immediately understand why their knowledge matters.
3. See a small set of topics worth discussing.
4. Press one calm Start button.
5. Begin talking naturally.
6. See an elegant waveform that confirms DutyGraph is listening.
7. See their words appear as a live transcript.
8. See DutyGraph quietly recognize topics already covered.
9. Receive an occasional optional prompt when important context appears missing.
10. Press Finish.
11. Review what DutyGraph understood.
12. Correct anything necessary.
13. Press Save & send.

Target feeling: “I just explained my work to a smart junior teammate.”

2. Core product thesis

DutyGraph should not require workers to document their own work. Workers should explain their work naturally. DutyGraph should perform the documentation.

Human speaks naturally → DutyGraph listens intelligently → AI structures the knowledge → human validates the result.

3. Why this matters strategically

This is not merely a nicer audio recorder. It may become the core ingestion mechanism for the Duty Graph. Traditional organizational discovery scales with interviewer hours. DutyGraph should compress this by allowing many participants to respond asynchronously and in parallel.

- Leadership establishes organizational context once.
- DutyGraph creates contextualized requests across the organization.
- Employees explain their work asynchronously.
- AI structures and compares the responses.
- Advisors review exceptions, disagreements, gaps, and high-value findings.

4. Design principle: Listen intelligently

- Listen more than you speak.
- Understand context while the person talks.
- Notice what has been covered.
- Notice what remains unclear.
- Do not interrupt productive thought.
- Do not force a questionnaire sequence.
- Only guide when guidance creates meaningful additional context.

5. Existing behavior to preserve

- private participant requests
- per-user request assignment
- due dates
- participant authentication
- microphone permission handling
- MediaRecorder audio capture
- local IndexedDB audio recovery
- local draft persistence
- uploaded audio evidence
- SHA-256 integrity checks
- resumable chunked audio upload
- participant transcript review
- task-card generation
- participant task-card approval/correction
- original audio retention
- evidence provenance
- request submission
- advisor-side review

6. Current workflow being replaced

Read questions → Acknowledge notice → Start recording → Finish clip → Save recording → Upload audio → Create transcript → Wait → Review transcript → Add transcript to written response → Create task cards → Review task cards → Send response

The new experience should hide implementation details such as recording, uploading, transcription, transcript insertion, extraction, and submission state.

7. New participant journey

Stage 1: Understand

Headline: Help us understand how your work really gets done. Supporting copy: Think of this as briefing a new teammate. Talk naturally. Real examples, frustrations, workarounds, and useful tangents help us understand the work better.

Stage 2: Talk

Primary CTA: Start. No red. No giant microphone. The page transitions in place to Listening with a responsive waveform and live transcript.

Stage 3: Review

Headline: Nice. Here's what we heard. Show coverage summary and extracted work understanding before the raw transcript.

Stage 4: Send

Primary CTA: Save & send. Confirmation thanks the participant and explains that their perspective will be combined with the rest of the team.

8. Participant intro copy

Things worth talking about:

- What usually starts the work
- What information you receive
- What you actually do
- Tools or systems you use
- People you depend on
- Decisions that require your judgment
- What happens when something goes wrong
- Who receives the result

Small disclosure: Private to this project · You can review before sending.

9. Start button design

- Do not use red.
- Do not use the word Record as the primary action.
- Do not display a large red microphone icon.
- Use a calm charcoal or off-white button with a thin neutral border.
- Use a subtle green border only on hover/focus/active.
- Primary label: Start.

10. Consent interaction

Preferred copy: DutyGraph will use your microphone to create a live transcript of your response. Your advisor receives your response only after you choose Save & send. You can review everything first. By selecting Start, you agree to provide this response and allow the audio to be processed for transcription.

If legal requires an explicit checkbox, keep it visually light and human. Do not style it like a warning.

11. Active listening screen

Listening



“Usually sales sends me the request first, and then I open NetSuite...”

We're following along

- ✓ How the work starts
- ✓ What information you receive
- What you actually do
- ✓ Systems you use

- Decisions that need your judgment
- What happens when things go wrong

Pause

Finish

12. Waveform / equalizer

- Drive locally using Web Audio API.
- Use `AudioContext` → `MediaStreamAudioSourceNode` → `AnalyserNode` → `requestAnimationFrame`.
- Respond immediately after microphone permission succeeds.
- Remain independent of the transcription service.
- Use 24-48 restrained bars, low height, subtle opacity variation.
- Support `prefers-reduced-motion`.

13. Live transcript behavior

Maintain two transcript states: partial transcript for immediate visual feedback and committed transcript for stable segments. Only committed transcript is analyzed by the coverage engine.

14. Speech-to-text architecture

V1 recommendation: OpenAI GPT-Live-Transcribe over WebRTC. Use it as the realtime transcription layer while retaining the existing batch transcription path as fallback. Do not make GPT-Live-1 the default V1 transcription engine; reserve it for an optional future facilitator.

Browser microphone

- └─ local waveform
- └─ local `MediaRecorder` backup
- └─ WebRTC → OpenAI live transcription → transcript deltas → `DutyGraph` session state

15. Preserve local audio recording

Realtime transcription must not replace local evidence capture. Continue `MediaRecorder` + `IndexedDB` + checksum + resumable upload. The waveform is UX feedback, realtime STT is the interaction layer, and the original audio is evidence/recovery.

16. Automatic upload behavior

Remove visible Save recording and manual upload from the normal flow. Upload silently during or immediately after the session. Only surface progress if recovery requires it.

17. Domain vocabulary injection

Before opening the realtime session, build a terminology package from company name, participant name, role, department, software, customers, acronyms, workflows, vendors, and named people. Use it to improve recognition of enterprise-specific terms.

18. Topic coverage system

Every work-discovery request should contain a hidden coverage rubric in addition to visible prompts.

- Purpose
- Trigger

- Inputs
- Input source
- Main actions
- Sequence
- Systems/software
- Dependencies
- Human decisions
- Approvals
- Exceptions
- Failure modes
- Output
- Recipient/handoff
- Frequency/volume
- Frustrations or constraints
- Undocumented workarounds

19. Topic states

Internal states: unheard, emerging, covered, conflicted. Participant-facing states: ○ Not discussed, ◉ Mentioned, ✓ Covered. Never show conflict state live to the participant.

20. Coverage scoring

Use semantic evaluation, not keyword matching. Every covered item must include supporting transcript segment IDs. The model must never invent evidence.

```
{
  "id": "exceptions",
  "state": "covered",
  "confidence": 0.86,
  "evidence": ["If it's a Yamaha account, Steve has to approve it before I place the order."]
}
```

21. Coverage evaluation cadence

Do not analyze every partial word. Evaluate after 10-20 seconds, a meaningful silence, or 2-4 new committed utterances. Send newest stable text plus prior coverage state and only the context needed.

22. Participant coverage UI

Label: We're following along. Subtext: These update as you talk. You don't have to cover every item.

- Never use red for missing topics.
- Never display a completion percentage.
- Never use "failed."
- Never force every topic.
- Never interrupt someone because a topic is incomplete.

23. Gentle follow-up prompts

DutyGraph should first try to obtain context passively. After a natural pause, it may surface one optional prompt based on a meaningful uncovered rubric dimension.

- What usually happens when the normal process breaks down?
- Who usually needs what you produce?
- Are there decisions here that only you or a manager can make?
- Is there a system or spreadsheet you rely on that we haven't talked about?
- What is the frustrating part of this process?

24. Natural pause detection

- Minimum ~4 seconds of silence.
- Participant has spoken at least ~45 seconds.
- No prompt shown within the prior ~60 seconds.
- A meaningful coverage gap exists.
- Only one prompt visible at a time.
- Dismiss once the participant resumes speaking.

25. Active-session controls

Only two dominant controls: Pause and Finish. When paused, use Continue. Avoid Stop recording / Resume recording language.

26. Finish behavior

14. Stop realtime audio transmission.
15. Finalize the current transcript segment.
16. Finalize MediaRecorder.
17. Upload remaining local audio.
18. Run final coverage reconciliation.
19. Run work extraction.
20. Transition directly into review.

27. Review experience

Headline: Nice. Here's what we heard. Show You covered and You may want to add. Controls: Add more and Looks good. Raw transcript should be collapsed by default.

28. Work understanding review

The primary review artifact should be human-readable work cards with Starts when, You usually, Uses, Needs judgment when, and Result. Actions: Looks right / Change something.

29. Final submission

Primary CTA: Save & send. Confirmation copy: You just helped us understand a part of the company that often lives only in people's heads.

30. Tone rules

- We want your expertise.
- You understand this work better than the software does.
- AI is helping document what you know.

- Avoid surveillance, performance evaluation, redundancy, replacement, or compliance-interrogation language.

31. Brand / aesthetic

- warm
- quiet
- competent
- enterprise-grade
- premium
- human
- patient
- unobtrusive

Avoid glowing AI imagery, robotic motifs, red indicators, AI avatars, bright gradients, gamification, completion scores, confetti, or “AI magic” language.

32. Mobile-first requirements

- One-column layout.
- Start reachable quickly.
- Waveform width fits viewport minus 32px.
- Sticky active controls near bottom.
- Safe-area support on iOS.
- Avoid viewport jumping as transcript grows.
- Recover local state after interruption.
- Test iPhone Safari, Android Chrome, mobile Gmail handoff, AirPods/Bluetooth, and noisy environments.

33. Frontend component structure

```
Participant.tsx
├── WorkCaptureExperience.tsx
│   ├── CaptureIntro.tsx
│   ├── CaptureConsent.tsx
│   ├── LiveCaptureSession.tsx
│   │   ├── AudioWaveform.tsx
│   │   ├── LiveTranscript.tsx
│   │   ├── CoveragePanel.tsx
│   │   ├── GentlePrompt.tsx
│   │   └── CaptureControls.tsx
│   ├── CaptureReview.tsx
│   │   ├── CoverageSummary.tsx
│   │   ├── TranscriptDisclosure.tsx
│   │   └── WorkUnderstandingReview.tsx
│   └── CaptureComplete.tsx
```

Hooks/services:

```
useMicrophoneCapture.ts
useRealtimeTranscription.ts
useAudioRecovery.ts
useCoverageAnalysis.ts
useCaptureSession.ts
```

34. Realtime transcription provider abstraction

```
interface RealtimeTranscriptionProvider {  
  connect(config): Promise<void>;  
  start(stream): Promise<void>;  
  pause(): Promise<void>;  
  resume(): Promise<void>;  
  finish(): Promise<void>;  
  disconnect(): Promise<void>;  
  onPartial(cb): void;  
  onCommitted(cb): void;  
  onError(cb): void;  
}
```

Initial implementation: OpenAIIiveTranscriptionProvider. Preserve the option to add Deepgram, AssemblyAI, or ElevenLabs later for benchmarking.

35. Backend additions

- POST /v1/participant/requests/:requestId/capture-session
- POST /v1/participant/requests/:requestId/coverage
- POST /v1/participant/requests/:requestId/capture-checkpoint
- POST /v1/participant/requests/:requestId/capture-complete

The backend must validate participant assignment, request state, tenancy, provider configuration, and consent; return only short-lived realtime authorization/session material; and preserve existing evidence semantics.

36. Suggested data model

```
CaptureSession { id, tenantId, companyId, requestId, participantId, state, startedAt, finishedAt, transcriptRevision,  
transcriptProvider, transcriptModel, audioAssetId, coverageRubricVersion, coverageRevision, createdAt, updatedAt }
```

```
TranscriptSegment { id, captureSessionId, index, text, startedAtMs, endedAtMs, providerFinal }
```

```
CoverageItem { key, label, description, state, confidence, evidenceSegmentIds[] }
```

37. Separate coverage from work extraction

```
speech → transcript → coverage → final transcript → work extraction → participant validation
```

The coverage evaluator only determines whether requested evidence exists. It should not also generate duties/tasks. Keep responsibilities separated.

38. Coverage engine contract

Evaluate only whether the supplied transcript contains meaningful evidence for each provided coverage dimension. Do not infer missing facts. A topic is covered only when explicit transcript evidence supports it. Return supporting segment IDs. Prefer emerging when evidence is incomplete.

39. GPT-Live-1 phase

Support GPT-Live-1 later as an optional restrained facilitator. Desired behavior: 90% listener / 10% facilitator. It may speak only when the participant asks something, appears stuck, a major gap remains after a natural pause, a critical ambiguity requires clarification, or the participant asks what else to cover. Never interrupt productive speech.

40. Resilience

If realtime transcription disconnects, continue local MediaRecorder, keep waveform active, show: Live text paused. Keep talking. Your response is still being saved. Attempt reconnection. After Finish, use batch transcription as fallback if necessary.

41. Privacy and trust

- Explain who receives the response.
- Explain why it is requested.
- Explain how audio is used.
- Explain whether audio is retained.
- Explain when an external AI provider is used.
- Let participants review before sending.
- Maintain the distinction between work understanding and company authority/permissions.

42. Accessibility

- Keyboard-operable Start/Pause/Finish.
- Clear focus states.
- Screen-reader state announcements.
- Waveform not required to understand status.
- Reduced motion support.
- WCAG contrast.
- Large touch targets.
- Status cannot depend on color alone.

43. Analytics

- capture_link_opened
- capture_started
- microphone_permission_granted/denied
- realtime_connected
- first_transcript_delta
- coverage_first_update
- gentle_prompt_shown/followed
- capture_paused/resumed
- capture_finished
- review_opened
- transcript_expanded
- add_more_started
- work_card_corrected
- capture_submitted
- capture_abandoned

44. North-star metric

Validated organizational context captured per employee-minute

DutyGraph is trying to compress organizational discovery. Measure useful + validated + structured information divided by participant effort.

45. Performance targets

- Start → waveform: <300 ms after permission success.
- Speech → partial transcript: target <750 ms perceived latency.
- Committed transcript → coverage update: target <5 seconds.
- Finish → review: target <8 seconds on a healthy network.

46. V1 implementation plan

Phase 1: UX foundation

- Refactor participant capture UI.
- Build CaptureIntro, CaptureConsent, LiveCaptureSession, AudioWaveform, CaptureControls, CaptureReview.
- Replace Start recording with Start and Finish clip with Finish.
- Hide Save recording / Create transcript / Use transcript in normal flow.

Phase 2: Realtime transcription

- Add useRealtimeTranscription.
- Implement OpenAILiveTranscriptionProvider using GPT-Live-Transcribe over WebRTC.
- Support partial/final segments, terminology hints, reconnect, provider errors.
- Preserve batch transcription fallback.

Phase 3: Audio recovery

- Reuse MediaRecorder, IndexedDB, resumable upload, checksums.
- Ensure network loss cannot destroy a response.

Phase 4: Coverage engine

- Create standardized coverage schema.
- Generate request-specific rubric.
- Analyze committed transcript periodically.
- Start with trigger, inputs, actions, systems, dependencies, judgment, exceptions, output/handoff.

Phase 5: Gentle prompts

- No more than one prompt/minute.
- Natural pause required.
- Only high-value missing coverage.
- Participant can ignore.

Phase 6: Simplified review

- Show coverage summary and extracted work first.
- Keep transcript optional.
- Add more + Save & send.

Phase 7: Instrumentation

- Ship analytics with the feature.

- Do not wait until later to measure the discovery-speed thesis.

47. Current files likely affected

- client/src/capture.tsx
- client/src/TranscriptReview.tsx
- client/src/participant.css
- shared/request-capture.ts
- server/transcription.ts

Likely additions: WorkCaptureExperience.tsx, LiveCaptureSession.tsx, AudioWaveform.tsx, LiveTranscript.tsx, CoveragePanel.tsx, GentlePrompt.tsx, CaptureReview.tsx, useMicrophoneCapture.ts, useRealtimeTranscription.ts, useCoverageAnalysis.ts, capture-session.ts, capture-coverage.ts, and corresponding shared schemas.

48. Do not break

- tenant isolation
- participant/request authorization
- request expiration
- local draft recovery
- original audio evidence
- audit trail
- provenance
- idempotent submission
- advisor review
- participant correction
- task versioning
- current batch transcription fallback
- typed-response alternative

49. Error states

Microphone denied

We couldn't access your microphone. You can allow microphone access in your browser, type your response, or upload audio instead.

Realtime transcript unavailable

Live text is temporarily unavailable. Keep talking. Your response is still being saved.

Network offline

Connection interrupted. Keep this page open. Your response is being kept on this device and will continue when you reconnect.

Recovered draft

Welcome back. We recovered your unfinished response from this device.

Provider failure after finish

Your audio is safe. We're creating the transcript from your saved response.

50. Participant copy library

Intro: Help us understand how your work really gets done. Think of this as briefing a new teammate. Talk naturally. Real examples, frustrations, exceptions, and useful tangents are often the most helpful parts.

Start: Start

Active: Listening · Talk naturally. Take your time.

Coverage: We're following along · These update as you talk. You don't have to cover every item.

Prompt: If it helps...

Pause: Pause

Continue: Continue

End: Finish

Review: Nice. Here's what we heard.

Final CTA: Save & send

Confirmation: Thank you, [Name]. You just helped us understand a part of the company that often lives only in people's heads.

51. Tagline system

- Primary: Listen intelligently. Map the work.
- Participant-facing: We listen intelligently, so you can just talk.
- Methodology: Listen intelligently. Structure automatically. Validate with people.
- Strategic: Traditional discovery asks people to document their work. DutyGraph listens intelligently and builds the structure for them.

52. GPT-Live-1 experiment after V1

Run an A/B pilot: Group A uses silent DutyGraph with live transcription, coverage, and text prompts. Group B uses a restrained GPT-Live-1 facilitator. Compare completion, session length, coverage, participant comfort, interruptions, corrections, extra useful facts, and abandonment. Do not assume spoken AI is better. Prove it.

53. Definition of done for V1

21. Participant understands the purpose in under 15 seconds.
22. One Start action begins the experience.
23. Microphone activity is obvious without red recording UX.
24. Speech appears as low-latency live text.
25. Audio remains recoverable if realtime transcription fails.
26. Coverage topics update semantically while the person speaks.
27. Missing topics can create unobtrusive optional prompts.
28. Finish automatically finalizes transcript and audio.
29. No manual upload or manual transcription request in the normal flow.
30. Participant reviews a human-readable work understanding.
31. Raw transcript remains optional.
32. Participant can add more context.
33. One Save & send action completes submission.

- 34. Advisor receives equal or better provenance than today.
- 35. Authorization and tenant tests continue to pass.
- 36. Mobile Safari and Chrome are reliable.
- 37. Analytics measure context captured per employee-minute.

54. Product principle to protect

Employees ramble. DutyGraph listens.

Do not let this evolve into an AI interviewer that talks constantly. The value is not asking more questions. The value is understanding more from fewer interruptions. That is what Listen intelligently should mean.